

Kotlin



История и основы языка



Разработка мобильных приложений на Android

Основы языка для Android

Что мы научимся читать и писать

- Переменные, типы, условия, циклы и функции
- Классы, объекты, свойства и конструкторы
- Nullable-типы и обработка отсутствующих значений
- Дженерики, коллекции и функции с лямбдами

Что такое Kotlin

- Статически типизированный язык общего назначения
- Поддерживает объектный и функциональный стиль
- Развивается как открытый проект под руководством JetBrains
- Работает с JVM, JavaScript, WebAssembly и нативными платформами



JETBRAINS

- Компания создаёт инструменты для разработчиков
- IntelliJ IDEA стала основой Android Studio
- Kotlin появился из потребностей собственной разработки

Краткая история Kotlin

Год	Событие
2010	JetBrains начинает разработку языка
2011	Публичное представление Project Kotlin
2016	Kotlin 1.0
2017	Официальная поддержка Kotlin в Android
2019	Google объявляет подход Kotlin-first для Android

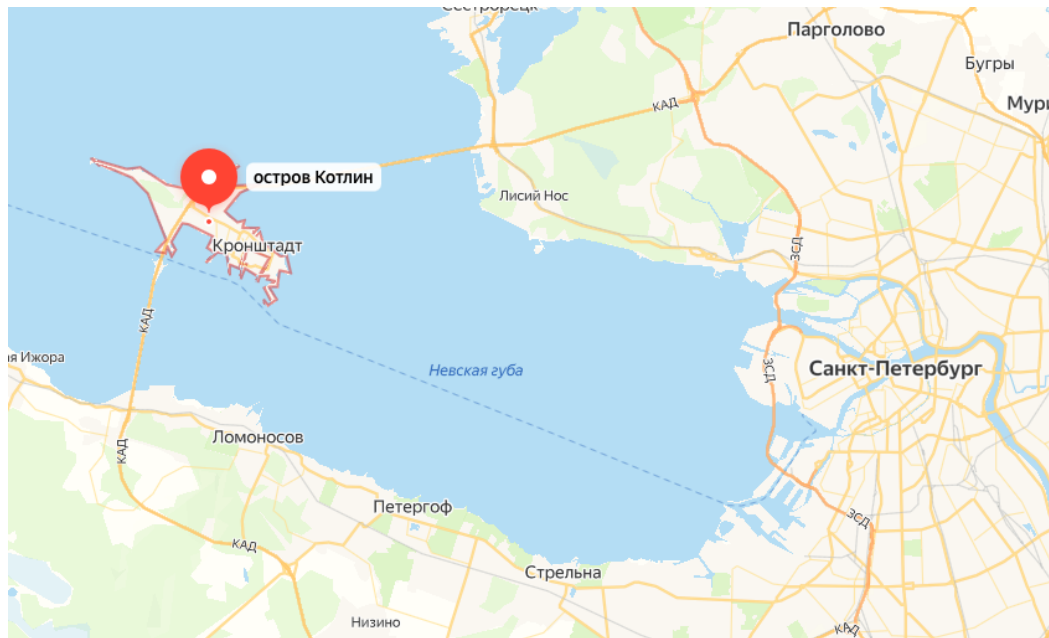
Почему появился ещё один язык

- JetBrains хотела современный язык для своих проектов на JVM
- Между Java 6, 7 и 8 проходили годы: 2006, 2011, 2014
- Нужны были статические проверки и удобный синтаксис
- Важно было сохранить существующий Java-код и библиотеки

Принципы проектирования Kotlin

Принцип	Что это означает на практике
Industry / interop	Встраиваться в реальные проекты и работать с Java
Tooling	Поддерживать навигацию, рефакторинг и подсказки IDE
Safety	Находить больше ошибок до запуска программы

Почему «Котлин»



Язык назван в честь острова Котлин в Финском заливе

Kotlin в нашем Android-курсе

На компьютере

Пишем исходники .kt в Android Studio

Kotlin/JVM компилирует код в class-файлы

Android-сборка преобразует байткод в DEX

На Android-устройстве

Устанавливаем приложение

ART исполняет код в формате DEX

Android вызывает компоненты приложения

А теперь сам язык

**Небольшие примеры.
Одна новая идея за раз.**



Kodee, маскот Kotlin

Hello, Kotlin!

```
fun main() {  
    println("Hello, Kotlin!")  
}
```

main — точка входа
консольной
программы.

println выводит
строку.

Файл Main.kt

Как читать первую программу

Фрагмент	Смысл
<code>fun</code>	Объявление функции
<code>main</code>	Имя функции
<code>()</code>	Здесь перечисляют параметры
<code>{ ... }</code>	Тело функции, выполняемые действия
<code>"Hello, Kotlin!"</code>	Строковое значение
<code>// комментарий</code>	Пояснение для человека, не выполняется

Выполняемые строки из следующих примеров размещаем внутри `main`.

Переменные: val и var

```
val name: String = "Латте"  
var cups: Int = 1
```

```
cups = 2  
println(name)  
println(cups)
```

```
// name = "Чай" // ошибка
```

val нельзя
переназначить.

var можно присвоить
новое значение.

Сначала выбираем val. Для изменения значения нужен var.

Вывод типов

```
val name = "Латте" // String  
val price = 180    // Int  
val hot = true     // Boolean
```

```
var cups = 2  
// cups = "два"    // ошибка
```

Компилятор выводит тип из выражения справа.

У переменной по-прежнему есть определённый тип.

Меньше записи не означает динамическую типизацию.

Типы для первых программ

Тип	Пример	Для чего
Int	180	Целое число
Long	3_000_000_000L	Целое число большего диапазона
Double	4.5	Дробное число
Boolean	true / false	Логическое условие
Char / String	'A' / "Латте"	Символ / строка

Числа и преобразование типов

```
val cups = 3
val total = 500

println(total / cups)    // 166
println(total.toDouble() / cups)
// 166.66666666666666

val count: Long = cups.toLong()
```

Int / Int даёт целое число.

Преобразование типа можно записать явно.

Присваивание Int переменной Long требует преобразования.

Строки и шаблоны

```
val name = "Латте"  
val cups = 2  
val price = 180  
  
println("Заказ: $name")  
println("Сумма: ${cups * price} ₺")  
println(name.length) // 5
```

\$name подставляет переменную.

В \${...} можно написать выражение.

String хранит текст. Сами строки неизменяемы.

Операторы и условия

Запись	Значение
+ - * / %	Арифметика, % — остаток от деления
> >= < <=	Сравнение чисел
== !=	Равенство и неравенство
&& !	И, ИЛИ, НЕ
=	Присваивание

Условие if

```
val cups = 2

if (cups > 0) {
    println("Готовим заказ")
} else {
    println("Заказ пуст")
}
```

В условии требуется Boolean.

Выполняется только выбранная ветка.

Фигурные скобки помогают видеть границы действий.

if может вернуть значение

```
val cups = 3

val delivery = if (cups >= 3) {
    0
} else {
    100
}

println(delivery) // 0
```

Результат последнего выражения выбранной ветки попадает в delivery.

Если if используется как выражение, нужна ветка else.

Несколько вариантов: when

```
val size = "M"

val volume = when (size) {
    "S" -> 200
    "M" -> 300
    "L" -> 400
    else -> 0
}

println(volume) // 300
```

when выбирает
подходящую ветку.

Стрелка отделяет
условие от
результата.

Здесь результат when имеет тип Int.

Цикл for и диапазоны

```
for (cup in 1..3) {  
    println("Стакан $cup")  
}  
  
for (index in 0 until 3) {  
    println(index)  
}
```

1..3 включает обе границы.

0 until 3 не включает 3.

Цикл повторяет тело для каждого значения.

Цикл while

```
var cups = 3

while (cups > 0) {
    println("Осталось: $cups")
    cups--
}
```

Проверка условия происходит перед каждой итерацией.

cups-- уменьшает значение на один.

Если забыть изменить условие, цикл может стать бесконечным.

break и continue

```
for (number in 1..5) {  
    if (number == 2) continue  
    if (number == 4) break  
  
    println(number)  
}  
// Вывод: 1, затем 3
```

continue пропускает остаток текущей итерации.

break завершает цикл.

return, который увидим в функциях, завершает вызов функции.

Вопрос: что выведет программа?

```
var total = 0

for (cup in 1..3) {
    total += 180
}

println(total)
println(if (total >= 500) "Бонус" else "—")
```

Сначала посчитайте значение total после каждой итерации.

30 секунд на ответ

Функция: имя, вход и результат

```
fun total(price: Int, cups: Int): Int {  
    return price * cups  
}
```

```
val amount = total(180, 2)  
println(amount) // 360
```

price и cups —
параметры.

180 и 2 — аргументы
вызова.

Int после скобок —
тип результата.

return передаёт результат вызывающему коду.

Короткая функция и Unit

```
fun total(price: Int, cups: Int): Int =  
    price * cups
```

```
fun printReceipt(amount: Int) {  
    println("Итого: $amount ₺")  
}
```

```
printReceipt(total(180, 2))
```

= удобно для одного выражения.

printReceipt имеет результат Unit.

Unit означает, что функция не возвращает полезный результат.

Имена и значения по умолчанию

```
fun total(  
    price: Int,  
    cups: Int = 1  
): Int = price * cups  
  
println(total(180))  
println(total(price = 180, cups = 2))
```

Если cups не передан,
используем 1.

Именованные
аргументы поясняют
смысл значений.

Результаты: 180 и 360

Область видимости и ранний return

```
fun printOrder(cups: Int) {  
    if (cups <= 0) {  
        println("Заказ пуст")  
        return  
    }  
    val message = "Готовим $cups"  
    println(message)  
}  
  
printOrder(0)
```

return без значения завершает Unit-функцию.

message доступна внутри её блока.

Переменные из тела функции снаружи не видны.

Отсутствующее значение: null

```
val name: String = "Латте"  
val comment: String? = null  
  
println(name.length)  
// println(comment.length) // ошибка
```

String требует строку.

String? допускает строку или null.

Вопросительный знак — часть типа.

Безопасный вызов и Elvis

```
val comment: String? = null

val length: Int? = comment?.length
val label: String = comment ?: "Без комментария"

println(length) // null
println(label)
```

?. вызывает член
только для
ненулевого значения.

?: выбирает запасное
значение при null.

Тип результата безопасного вызова обычно тоже nullable.

Проверка и smart cast

```
fun printComment(comment: String?) {  
    if (comment == null) return  
  
    println(comment.length)  
}  
  
printComment("Без сахара")
```

После проверки компилятор знает, что `comment` здесь не `null`.

Проверка имеет смысл, если значение не может незаметно измениться.

Что означает !!

```
fun lengthUnsafe(text: String?): Int {  
    return text!!.length  
}  
  
println(lengthUnsafe("Кофе")) // 4  
// lengthUnsafe(null)        // исключение
```

!! утверждает, что значение не null.

Если это не так, программа получает исключение.

!! переносит риск из проверки компилятора во время выполнения.

Число из пользовательского ввода

```
fun parseCups(text: String): Int? {  
    val cups = text.toIntOrNull() ?: return null  
    if (cups <= 0) return null  
    return cups  
}  
  
println(parseCups("2")) // 2  
println(parseCups("два")) // null
```

`toIntOrNull`
возвращает число
либо `null`.

Ранний `return`
обрабатывает
неудачу.

Для ожидаемой ошибки ввода можно вернуть nullable-результат.

Класс и объект

- Класс описывает данные и поведение
- Объект — конкретный экземпляр класса
- Свойства хранят или вычисляют данные объекта
- Методы — функции, принадлежащие объекту или классу

Первый класс Coffee

```
class Coffee(  
    val name: String,  
    val price: Int  
) {  
    fun label(): String = "$name: $price ₽"  
}
```

Coffee — имя класса.

name и price —
свойства.

label — метод.

Параметры с val или var сразу объявляют свойства.

Конструктор и создание объекта

```
val latte = Coffee("Латте", 180)
val espresso = Coffee("Эспрессо", 120)

println(latte.name)
println(espresso.price)
println(latte.label())
```

Вызов конструктора создаёт объект и задаёт его начальные данные.

Ключевое слово `new` не требуется.

Один класс, два независимых объекта

Проверка данных в init

```
class Coffee(  
    val name: String,  
    val price: Int  
) {  
    init {  
        require(price > 0)  
    }  
}
```

init выполняется при создании объекта.

require проверяет условие и сообщает об ошибке.

Объект с недопустимой ценой не должен успешно создаваться.

val не замораживает объект

```
class Cup(var temperature: Int)

val cup = Cup(80)
val sameCup = cup

cup.temperature = 60
println(sameCup.temperature) // 60

// cup = Cup(90) // ошибка
```

Переназначение val
запрещено.

Изменение var-
свойства объекта
разрешено.

Две переменные могут ссылаться на один объект.

Свойство и поле

```
class Order(val price: Int, val cups: Int) {  
    val total: Int  
        get() = price * cups  
}
```

```
val order = Order(180, 2)  
println(order.total) // 360
```

Свойство total
вычисляется при
чтении.

Отдельное поле для
него не нужно.

У свойства может быть getter, у var — ещё и setter.

Доступ к состоянию объекта

```
class Counter {  
    var value = 0  
    private set  
  
    fun increment() {  
        value++  
    }  
}
```

value можно
прочитать снаружи.

Изменять его
напрямую может
только сам Counter.

Изменение состояния проходит через метод.

Видимость объявлений

Модификатор	Область доступа
public	Везде, где доступно само объявление. По умолчанию
private	Внутри класса, а для верхнего уровня — в файле
protected	В классе и его наследниках
internal	В пределах модуля компиляции

data class для данных

```
data class Coffee(  
    val name: String,  
    val price: Int  
)  
  
val latte = Coffee("Латте", 180)  
val expensive = latte.copy(price = 200)  
  
println(expensive)
```

Компилятор создаёт
equals, hashCode,
toString и copy.

copy делает новый
объект.

Участвуют свойства первичного конструктора.

Равенство: == и ===

```
val a = Coffee("Латте", 180)  
val b = Coffee("Латте", 180)  
val c = a
```

```
println(a == b) // true  
println(a === b) // false  
println(a === c) // true
```

== использует equals.

=== проверяет, тот ли это объект.

Здесь Coffee объявлен как data class.

Интерфейс: общий контракт

```
interface Priced {  
    val price: Int  
}  
  
data class Coffee(  
    val name: String,  
    override val price: Int  
): Priced  
  
fun printPrice(item: Priced) = println(item.price)
```

Контракт Priced
требует свойство
price.

Coffee реализует этот
контракт.

Функция может работать с любым объектом типа Priced.

Наследование: open и override

```
open class Drink {  
    open fun label(): String = "Напиток"  
}  
  
class Tea : Drink() {  
    override fun label(): String = "Чай"  
}  
  
val drink: Drink = Tea()  
println(drink.label()) // Чай
```

По умолчанию классы и методы закрыты для наследования.

open разрешает переопределение.

Вызывается реализация фактического объекта.

Зачем нужны дженерики?

Как описать коробку, которая хранит кофе, строку или число?

Не хочется писать `CoffeeBox`, `StringBox`, `IntBox`...
Но хочется сохранить проверку типов.

Параметр типа T

```
class Box<T>(val value: T)

val coffeeBox: Box<Coffee> =
    Box(Coffee("Латте", 180))

val coffee: Coffee = coffeeBox.value
println(coffee.name)
```

T — параметр типа.

Coffee — конкретный аргумент типа.

Box<Coffee> читаем как «коробка с Coffee».

Что подставляется вместо T

Использование	Тип свойства value	Значение
Box<Coffee>	Coffee	Coffee("Латте", 180)
Box<String>	String	"Готово"
Box<Int>	Int	42

Вывод аргумента типа

```
class Box<T>(val value: T)

val message = Box("Готово") // Box<String>
val count = Box(42)         // Box<Int>

val text: String = message.value
// val number: Int = message.value // ошибка
```

Тип можно указать явно.

Часто компилятор выводит его из аргументов.

Угловые скобки описывают тип, круглые передают значения.

Обобщённая функция

```
fun <T> unpack(box: Box<T>): T {  
    return box.value  
}
```

```
val number: Int = unpack(Box(42))  
val text: String = unpack(Box("Готово"))
```

<T> перед именем
объявляет параметр
типа функции.

Тот же T связывает
вход и результат.

При каждом вызове T определяется из переданного Box.

Конкретный и обобщённый параметр

```
fun printCoffee(box: Box<Coffee>) {  
    println(box.value.name)  
}
```

```
fun <T> printBox(box: Box<T>) {  
    println(box.value)  
}
```

Первая функция принимает именно `Box<Coffee>`.

Вторая работает с разными `Box<T>`.

У произвольного `T` нельзя просто вызвать `.name`.

Простое ограничение типа

```
fun <T : Priced> printBoxPrice(box: Box<T>) {  
    println(box.value.price)  
}
```

```
val box = Box(Coffee("Латте", 180))  
printBoxPrice(box)
```

T : Priced требует,
чтобы тип
поддерживал
контракт Priced.

Теперь компилятор разрешает обратиться к price.

Коллекции: несколько объектов

Тип	Смысл	Пример
List<T>	Упорядоченные элементы, повторы возможны	Меню или заказы
Set<T>	Уникальные элементы	Теги напитка
Map<K, V>	Значение по уникальному ключу	Цена по названию

List<Coffee>: список кофе

```
val menu: List<Coffee> = listOf(  
    Coffee("Эспрессо", 120),  
    Coffee("Латте", 180)  
)
```

```
val first: Coffee = menu[0]  
println(first.name)  
println(menu.size) // 2
```

Coffee в угловых скобках задаёт тип элемента.

Индексы начинаются с нуля.

listOf создаёт список для чтения.

Изменяемый список

```
val cart: MutableList<Coffee> = mutableListOf()
```

```
cart.add(Coffee("Латте", 180))  
cart.add(Coffee("Эспрессо", 120))  
cart.removeAt(0)
```

```
println(cart.size) // 1
```

MutableList добавляет операции изменения.

Тип элементов по-прежнему Coffee.

val cart запрещает переназначение, но разрешает менять список.

List: доступ только для чтения

```
val source = mutableListOf("Латте")  
val menu: List<String> = source  
  
source.add("Чай")  
println(menu.size) // 2  
  
// menu.add("Какао") // недоступно
```

Через menu нельзя
добавить элемент.

Сам объект всё ещё
может меняться
через source.

Read-only интерфейс не гарантирует неизменность объекта.

Функция с параметром List<Coffee>

```
fun totalPrice(items: List<Coffee>): Int {  
    var total = 0  
    for (coffee in items) {  
        total += coffee.price  
    }  
    return total  
}
```

Параметр items — список объектов Coffee.

Переменная coffee внутри цикла имеет тип Coffee.

Эта функция принимает конкретный тип элементов.

Обобщённая функция для списка

```
fun <T> firstOrNull(items: List<T>): T? {  
    if (items.isEmpty()) return null  
    return items[0]  
}  
  
val name: String? = firstOrNull(listOf("Латте"))  
val number: Int? = firstOrNull(emptyList<Int>())
```

List<T> связывает тип элементов и тип результата T?.

Пустой список даёт null.

Это учебная реализация: в Kotlin уже есть list.firstOrNull().

Set и Map

```
val tags: Set<String> = setOf("hot", "hot")
println(tags.size) // 1
```

```
val prices: Map<String, Int> = mapOf(
    "Латте" to 180,
    "Чай" to 100
)
println(prices["Латте"]) // 180
println(prices["Какао"]) // null
```

Set убирает повторы.

Map связывает ключ со значением.

Поиск отсутствующего ключа даёт null.

В mapOf запись key to value создаёт пару.

Массив `Array<T>`

```
val names: Array<String> = arrayOf("Латте", "Чай")
names[0] = "Kakao"

println(names[0]) // Kakao
println(names.size) // 2
```

Размер массива фиксирован после создания.

Элементы можно заменить.

Для обычного набора данных чаще удобнее `List` или `MutableList`.

Функцию можно хранить в переменной

```
val double: (Int) -> Int = { value ->
    value * 2
}

println(double(3)) // 6
```

(Int) -> Int — тип функции.

Справа лямбда: параметр и тело.

Последнее выражение — результат лямбды.

Функции в Kotlin — значения первого класса.

Функция принимает другую функцию

```
fun transform(  
    value: Int,  
    operation: (Int) -> Int  
): Int = operation(value)  
  
val result = transform(3, { n -> n * 2 })  
println(result) // 6
```

operation — обычный параметр, у которого функциональный тип.

Функция вызывает переданное действие.

Лямбда после круглых скобок

```
val a = transform(3, { n -> n * 2 })  
val b = transform(3) { n -> n * 2 }  
val c = transform(3) { it * 2 }
```

```
println(a == b && b == c) // true
```

Последний аргумент-лямбду можно вынести за скобки.

`it` — неявное имя единственного параметра.

Все три вызова передают одинаковую операцию.

filter и map

```
val menu = listOf(  
    Coffee("Эспрессо", 120),  
    Coffee("Латте", 180)  
)  
  
val cheap: List<Coffee> =  
    menu.filter { it.price <= 150 }  
  
val names: List<String> =  
    cheap.map { it.name }
```

filter оставляет
подходящие
элементы.

map преобразует
каждый элемент.

Исходный список menu не меняется.

Частые операции с коллекциями

```
val sum = menu.sumOf { it.price }  
val hasCheap = menu.any { it.price < 150 }  
val latte = menu.find { it.name == "Латте" }  
val sorted = menu.sortedBy { it.price }  
  
println(sum)      // 300  
println(hasCheap) // true  
println(latte?.name)
```

sumOf считает сумму.

any проверяет
наличие.

find возвращает
элемент или null.

sortedBy возвращает новый отсортированный список.

Одна задача: цикл или операции

```
fun totalPrice(items: List<Coffee>): Int =  
    items.sumOf { coffee -> coffee.price }  
  
val total = totalPrice(menu)  
println(total) // 300
```

Мы заменили ручное накопление суммы одной библиотечной операцией.

Краткую запись стоит использовать, когда её смысл понятен.

Функции-расширения

```
fun Coffee.isAffordable(budget: Int): Boolean {  
    return price <= budget  
}
```

```
val latte = Coffee("Латте", 180)  
println(latte.isAffordable(200)) // true
```

Coffee перед точкой
— тип получателя.

Внутри можно
обращаться к его
доступным членам.

Расширение не меняет исходное определение класса.

let для nullable-значения

```
val coffee: Coffee? = menu.find {  
    it.name == "Латте"  
}  
  
coffee?.let { found ->  
    println(found.name)  
    println(found.price)  
}
```

Если coffee не null, let выполнит блок.

found внутри имеет тип Coffee.

Обычный if (coffee != null) здесь тоже подходит.

apply для настройки объекта

```
class Cup {  
    var volume = 0  
    var hasLid = false  
}  
  
val cup = Cup().apply {  
    volume = 300  
    hasLid = true  
}
```

Внутри apply this обозначает настраиваемый объект.

Результат apply — тот же объект.

Полезно для нескольких связанных настроек.

enum: фиксированный набор вариантов

```
enum class Size {  
    SMALL, MEDIUM, LARGE  
}  
  
fun volume(size: Size): Int = when (size) {  
    Size.SMALL -> 200  
    Size.MEDIUM -> 300  
    Size.LARGE -> 400  
}
```

Вместо произвольной строки используем один из известных вариантов.

Когда перечислены все варианты enum, else здесь не нужен.

object: единственный экземпляр

```
object MenuDefaults {  
    const val MAX_CUPS = 10  
}  
  
fun isAllowed(cups: Int): Boolean =  
    cups in 1..MenuDefaults.MAX_CUPS
```

object объявляет
именованный объект.

Создавать его через
MenuDefaults() не
нужно.

Общее изменяемое состояние требует осторожности.

companion object внутри класса

```
class Coffee(val name: String, val price: Int) {  
    companion object {  
        fun default(): Coffee = Coffee("Латте", 180)  
    }  
}  
  
val coffee = Coffee.default()  
println(coffee.name)
```

companion object
хранит объявления,
связанные с классом.

К функции обращаемся через имя класса.

Исключения: try и catch

```
fun parseStrict(text: String): Int {  
    return try {  
        text.toInt()  
    } catch (error: NumberFormatException) {  
        0  
    }  
}  
  
println(parseStrict("два")) // 0
```

try выполняет код.

catch обрабатывает конкретный тип исключения.

Для простого ввода toIntOrNull обычно понятнее.

Файлы, package и import

```
package ru.example.coffee

import kotlin.math.max

fun main() {
    val cups = max(1, 0)
    println(cups)
}
```

package задаёт пространство имён.

import позволяет использовать короткое имя объявления.

В одном .kt-файле могут быть несколько функций и классов.

Частые ошибки в первых программах

Ошибка	Как рассуждать
Ставить !! везде	Определить поведение при null
Считать val полной неизменяемостью	Разделять ссылку и состояние объекта
Считать List изменяемым списком	Проверить контракт List / MutableList
Терять смысл за цепочкой it	Дать параметрам имена или использовать цикл
Копировать <T> как заклинание	Назвать тип входа, элемента и результата

Мини-задача: меню кофейни

**Показать названия напитков,
которые подходят под бюджет**

Есть List<Coffee> с названием и ценой.
На вход приходит бюджет строкой.
Неверный ввод нужно обработать.

Условия и проверочные примеры

Дано

Меню: Эспresso 120, Латте 180, Чай 100

Бюджет "150"

Бюджет "50"

Бюджет "два" или "-10"

Ожидаем

Порядок исходного меню сохраняется

[Эспresso, Чай]

Пустой список

Сообщение о некорректном бюджете

Возможное решение: функция

```
fun affordableNames(  
    menu: List<Coffee>,  
    budget: Int  
): List<String> {  
    return menu  
        .filter { it.price <= budget }  
        .map { it.name }  
}
```

List<Coffee> на входе.

List<String> на
выходе.

Логика выбора отделена от чтения пользовательского ввода.

Возможное решение: проверка ввода

```
fun showMenu(menu: List<Coffee>, input: String) {  
    val budget = input.toIntOrNull()  
    if (budget == null || budget < 0) {  
        println("Некорректный бюджет")  
        return  
    }  
    println(affordableNames(menu, budget))  
}
```

После проверки
budget имеет
ненулевой тип Int.

Пустой результат
остаётся обычным
списком.

Некорректный ввод и отсутствие подходящих напитков — разные ситуации.

Демонстрация после лекции

- Создать Coffee и заполнить меню
- Пройти от обычного цикла к `filter` и `map`
- Изменить тип элемента и прочитать ошибку компилятора
- Проверить пустое меню и некорректный ввод
- Запустить программу с точкой останова и посмотреть значения

Что нужно унести с занятия

- Типы помогают описывать допустимые значения и операции
- Класс связывает данные с поведением, объект хранит состояние
- T позволяет сохранить связь между типами входа и результата
- `List<Coffee>` означает список элементов типа `Coffee`
- Лямбда передаёт действие, которое выполнит другая функция

Источники и дальнейшее чтение

- История и изображения: исходная презентация преподавателя
- Kotlin Tour: kotlinlang.org/docs/kotlin-tour-welcome.html
- Основы: kotlinlang.org/docs/basic-syntax.html
- Дженерики: kotlinlang.org/docs/generics.html
- Android и Kotlin: developer.android.com/kotlin/learn